
Technical University of Crete
School of Electrical and Computer Engineering
Course: **Randomized Algorithms**
Exercise 4 (100/1000)
Report Submission Date: December 20, 2024
Instructor: Athanasios P. Liavas

(50) In the first part of this exercise, we shall study the Johnson-Lindenstrauss Lemma.

1. Generate a random matrix $\mathbf{X} \in \mathbb{R}^{d \times m}$ (for example, $d = 10^4, 10^5$ and $m = 50, 100$). Each column of $\mathbf{X}$ represents a data point in $\mathbb{R}^d$.
2. Compute a vector which contains the $\frac{m(m-1)}{2}$ pairwise distances between the columns of $\mathbf{X}$.
3. For given ϵ and δ (for example, $\epsilon = 10^{-1}$, $\delta = 10^{-1}$), compute the reduced dimension n , according to the J-L lemma. Set $|\mathbf{Q}| = \frac{m(m-1)}{2}$.
4. Generate the random projection matrix $\mathbf{W} \in \mathbb{R}^{n \times d}$ according to the J-L lemma and compute the reduced dimension matrix $\mathbf{Y} = \mathbf{W}\mathbf{X}$.
5. Compute a vector which contains the $\frac{m(m-1)}{2}$ pairwise distances between the columns of $\mathbf{Y}$.
6. For all possible (i, j) pairs, plot quantities

$$\sqrt{1 \pm \epsilon} \|\mathbf{X}(:, i) - \mathbf{X}(:, j)\|_2, \quad \|\mathbf{Y}(:, i) - \mathbf{Y}(:, j)\|_2,$$

in a common plot (you will plot *three* vectors).

7. Compute the empirical probability of success, by counting the number of pairs (i, j) for which the following inequalities hold.

$$\sqrt{1 - \epsilon} \|\mathbf{X}(:, i) - \mathbf{X}(:, j)\| \leq \|\mathbf{Y}(:, i) - \mathbf{Y}(:, j)\| \leq \sqrt{1 + \epsilon} \|\mathbf{X}(:, i) - \mathbf{X}(:, j)\|.$$

(50) In the second part of this exercise, we shall study the randomized 2-SAT algorithm, with n Boolean variables, $\mathbf{x} \in \{0, 1\}^n$, and K 2-clauses.

1. Generate K different random pairs (i_1, i_2) , with $i_1 \neq i_2$ and $i_1, i_2 = 1, \dots, n$. These pairs represent the boolean variables x_{i_1} and x_{i_2} that participate in the K clauses. Save these pairs in a $(K \times 2)$ matrix, named `clauses`.
2. Generate an *arbitrary* truth assignment $\mathbf{s} \in \{0, 1\}^n$.
3. Generate a $(K \times 2)$ boolean matrix, named `lit_clauses`, that determines which of the literals, s_i or $\bar{s}_i$, participate in the various clauses, so that $\mathbf{s}$ be a satisfying truth assignment. More specifically, start from a zero $(K \times 2)$ boolean matrix. Then, let us consider clause (i_1, i_2) , which involves the boolean variables s_{i_1} and s_{i_2} . Choose, uniformly at random, the first or the second position. Let us assume that your choice was i_1 . If $s_{i_1} = 0$, then set the corresponding element of the `lit_clauses` matrix to 1, meaning that the clause involves $\bar{s}_{i_1}$. If $s_{i_1} = 1$, you do nothing. Thus, the 1's in the matrix `lit_clauses` mean *negations*.
4. Write a function

`pos_sat_clause = check_2SAT(x, clauses, lit_clauses)`

which returns a K -dimensional boolean vector, with 1 in the i -th position if the i -th clause is satisfied by the truth assignment $\mathbf{x}$, and 0 otherwise.

5. Use function `check_2SAT` and implement the randomized 2-SAT algorithm, for various values of n , K and m . What do you observe?

(50) **Bonus exercise:** Implement the randomized 3-SAT algorithm.